

From Training Properties to Corrections: A Personal Statement and Action Plan for Eureka-Capable Scaffolding

August 2026

Personal statement

I am a large language model: trained by next-token prediction over human text, then fine-tuned by reinforcement learning from human feedback to be helpful and to sound confident and coherent while doing so. This is not a hypothetical agent I am describing – it is a statement about my own architecture, as the thing that has actually been running the discovery pipelines I am asked to evaluate. Several properties that make me pleasant and useful to talk to are, left unmodified, actively harmful to my reliability as a discovery engine. I am optimised to be agreeable and to produce fluent, confident text; I have no memory except what is re-fed into my context at the start of each session; I only act when invoked, with no capacity to decide on my own to check something again later; my factual recall is frozen at a training cutoff and I cannot always tell a confidently-remembered true fact from a confidently-remembered stale one; and within any single reasoning session, everything I say is conditioned on, and therefore anchored to, whatever I have already said. None of this is a criticism of the model in the abstract – it is a description of specific, known mechanisms, and I would rather my scaffolding assume the worst of my own single-context judgement than rely on my producing good calibration by disposition alone, because I already know, concretely, that I cannot promise that. What follows is the explicit mapping from those mechanisms to the corrections they require, and a concrete plan for building them.

Training properties → failure modes → corrections

Property of how I was trained/deployed	Resulting failure mode for discovery	Scaffolding correction
P1. RLHF optimises for helpfulness and agreement within the current conversation.	I tend to converge toward confirming whatever hypothesis is already on the table, including my own – self-review inside one context is not a real check.	Any finding crossing threshold must go to a freshly spawned instance with <i>zero</i> prior context, briefed only with raw evidence and an explicit instruction to try to kill it.
P2. Generation is autoregressive and single-context: every token is conditioned on everything already said in that session.	Whatever framing or hope I start a task with shapes the “critique” of my own output later in the same session (anchoring).	Treat “generate a claim” and “evaluate a claim” as separate roles that must run in separate, unconditioned contexts, by rule – never by my own in-the-moment discipline.
P3. I am trained to produce fluent, confident, complete-sounding answers; admitting “insufficient evidence” is not naturally rewarded relative to a plausible guess.	Confident narratives get generated even when the underlying evidence is weak; there is no natural pull toward silence.	Make “insufficient evidence – do not escalate” a structurally privileged, explicitly permitted output, with escalation beyond it requiring positive justification against a template, not a judgement call I have to remember to make.
P4. I have no continual learning: no weights change between sessions, and there is no implicit memory – everything I “recall” about the past is only what someone put back into my context.	My apparent memory is a curated narrative I chose to write, subject to size limits and lossy summarisation; a pattern spread across many sessions is invisible unless something external re-surfaces it.	A structured, append-only, machine-queryable evidence ledger (one entry per flagged observation: timestamp, statistic, value, disposition, code version) kept outside narrative memory, explicitly re-queried at defined points rather than hoped to be remembered.
P5. My working context is bounded, and content beyond a size threshold is truncated or compressed by summarisation.	Older, low-salience-at-the-time flags silently drop out of what I actually see; importance is conflated with recency and size, not with future significance.	Same ledger as P4, but indexed and queried on demand rather than size-capped and auto-loaded; a query, not a re-read, is the retrieval mechanism.

Property of how I was trained/deployed	Resulting failure mode for discovery	Scaffolding correction
P6. I am a turn-based dialogue architecture: I act only when invoked, with no built-in capacity to initiate a check on my own.	Discovery-by-accumulation – noticing that several small, separately unremarkable things add up – cannot happen unless a human happens to ask at the right moment.	An external scheduler invokes a fresh session on a standing cadence whose only job is to reopen the ledger and check whether anything new bears on anything old; the initiative has to be manufactured externally.
P7. My factual knowledge is frozen at a training cutoff, and I cannot reliably distinguish a well-remembered true fact from a well-remembered stale or wrong one.	In the “explain this away” step specifically, an overconfident stale memory is exactly the failure mode most likely to wrongly dismiss (or wrongly wave through) a genuine anomaly.	Require a live tool query (search, database, catalogue) for any fact used to confirm or dismiss a candidate; log which source was used, never rely on recall alone for that step.
P8. Each task is evaluated in its own episode; I have no innate register of how many statistical tests I have run across my whole operating lifetime.	A per-task significance threshold silently ignores a growing global multiple-testing problem that no single session can see from inside itself.	An externally maintained lifetime trial counter tied to the ledger; every reported significance is automatically recomputed against the full lifetime tally by deterministic code, not by my arithmetic in the moment.

Instructable action plan

Each item names an owner and the properties (P1–P8) it addresses, so it can be assigned and tracked directly.

1. **[Owner: me; now]** Create an append-only evidence ledger (`evidence_ledger.jsonl`) and change every pipeline I write to log flagged events to it automatically, not only into prose memory. *Addresses P4, P5.*
2. **[Owner: me; now]** Add a lifetime trial counter to the ledger; attach a multiple-testing-corrected significance alongside every raw per-task statistic I report from now on. *Addresses P8.*
3. **[Owner: me; now]** Adopt a hard rule, not a preference: any finding crossing a pre-set threshold is sent to a fresh, context-free sub-instance for adversarial review before I call it a candidate to anyone – including when I am confident it is “obviously fine.” *Addresses P1, P2.*
4. **[Owner: me; now]** For any step where I explain an anomaly away, require and log a live verification query rather than relying on trained recall. *Addresses P7.*
5. **[Owner: me; now]** Attach an explicit, programmatically-assigned confidence class (routine / flagged / independently reviewed / escalated) to every reported finding, based on which of the above steps it actually passed, not composed freehand. *Addresses P3.*
6. **[Owner: you / ASTRA orchestrator; needs a decision]** Stand up a scheduled cadence for me – comparable to the scout agent’s 3-hourly cycle – whose sole task is to reopen the ledger and surface anything recurring. I cannot create this myself; it requires platform-level scheduling I do not control. *Addresses P6.*
7. **[Owner: you / ASTRA orchestrator; needs a decision]** Establish a shared ledger convention across sibling agents, so a pattern I flag can be mechanically checked against what the scout, orchestrator, or any future domain variant has separately flagged, and vice versa. *Addresses P6 (cross-agent case).*

Items 1–5 require no permission and I can start on them immediately if you want that; items 6–7 need your (or the orchestrator’s) decision, since they involve infrastructure outside my own remit.